





*Serviço Nacional
de Aprendizagem
Industrial*

Construtores, Destrutores e Encapsulamento

Programação de Aplicativos

Construtores, Destrutores e Encapsulamento

- **Instrutor:** Lucas Ribeiro Arêas
- **Unidade Curricular:** Programação de Aplicativos
- **Curso:** Técnico em Desenvolvimento de Sistemas

SENAI Vitória - DR/ES

1 - Construtores

Construtores

Como estudamos, sempre que queremos criar um novo objeto de uma determinada classe utilizamos a palavra ***new*** acompanhada por um construtor.

Construtores

O construtor de uma classe tem, por definição, o mesmo nome que a classe. A seguir veremos dois exemplos de construtores para a classe **Conta** que utilizamos em exemplos da aula anterior.

Construtores

```
public Conta(int numero, String nome_titular, double saldo){  
    this.numero = numero;  
    this.nome_titular=nome_titular;  
    this.saldo = saldo;  
}  
  
public Conta(int numero, String nome_titular){  
    this.numero = numero;  
    this.nome_titular=nome_titular;  
    saldo = 0;  
}
```

Construtores

É possível definir diversos construtores para a mesma classe, tendo os tipos ou a quantidade de parâmetros diferentes para cada um deles.

Construtores

O primeiro construtor do exemplo que vimos anteriormente (slide 7) cria objetos já atribuindo aos mesmos valores passados como parâmetros para os atributos **numero**, **nome_titular** e **saldo**.

Construtores

Já o segundo construtor recebe como parâmetros valores apenas para os atributos **numero** e **nome_titular** e atribui o valor zero ao **saldo**. Note a utilização da palavra reservada *this* para diferenciar os atributos dos parâmetros de mesmo nome.

Construtores

O exemplo a seguir exhibe o código de um programa que cria dois objetos da classe **Conta**, utilizando cada um dos construtores apresentados no exemplo anterior (slide 7).

Construtores

```
package exemplos;

public class Programa {

    public static void main(String[] args) {
        Conta c1, c2;
        c1 = new Conta(1, "Zé", 0);
        c2 = new Conta(2, "João");
    }
}
```

Construtores

Note que na criação do objeto c1 foi chamado o construtor que recebe três parâmetros enquanto na criação do objeto c2 foi chamado o outro construtor que recebe apenas dois parâmetros.

Construtores

A conta referenciada por c1 terá como o valor **1** para o atributo **numero**, o valor **Zé** para o atributo **nome_titular** e o valor **zero** para o **saldo**.

Construtores

Java define um construtor padrão para classes que não tem nenhum construtor definido. O construtor padrão não recebe nenhum argumento.

Construtores

No exemplo que usamos nas nossas aulas sobre Introdução a Orientação a Objetos, a classe Conta não tinha construtores definidos, o Java então utilizou o construtor padrão para instanciar os objetos.

Construtores

Porém, a partir do momento em que você declara um construtor, o construtor padrão deixa de existir. Assim, agora que criamos esses dois construtores é impossível criar uma nova instância de Conta sem utilizar um dos dois.

2 - Destrutores

Destrutores

Quando estamos programando, sempre devemos tomar cuidado com o espaço da memória que está utilizado.

Destrutores

Tudo que foi alocado dinamicamente em memória e não está sendo utilizado mais no código deve ser desalocado, e em muitas linguagens isso é feito manualmente.

Destrutores

Em Java isso não é necessário, pois essa linguagem possui um Coletor Automático de Lixo (*Garbage Collector*), o qual é responsável por desalocar tudo aquilo que não é mais utilizado. Assim, os objetos que não são mais referenciados por um programa são automaticamente desalocados por esse coletor, liberando memória.

Destrutores

Em Java, o método destrutor de uma classe é o método *finalize*. Quando não é definido um método destrutor para uma classe, Java utiliza um método destrutor padrão que não faz nada.

Destrutores

O método *finalize* é raramente utilizado porque pode causar problemas de desempenho e há uma incerteza sobre se ele será chamado.

3 - Encapsulamento

Encapsulamento

Encapsulamento é uma técnica utilizada para restringir o acesso a variáveis (atributos), métodos ou até à própria classe.

Encapsulamento

Os detalhes da implementação ficam ocultos ao usuário da classe, ou seja, o usuário passa a utilizar os métodos de uma classe sem se preocupar com detalhes sobre como o método foi implementado internamente.

Encapsulamento

Para facilitar o entendimento, façamos uma analogia com um carro. Para dirigir um carro uma pessoa não precisa conhecer os detalhes sobre como funciona o motor ou os demais componentes dele.

Encapsulamento

Um motorista não precisa saber o que acontece internamente no carro quando ele acelera ou quando troca de marcha. Para dirigir ele precisa apenas saber como dirigir o carro utilizando pedais de acelerador, freio e embreagem, volante e alavanca de câmbio.

Encapsulamento

Esses componentes encapsulam toda a complexidade existente no carro sob a ótica do motorista que o utiliza.

Encapsulamento

A ideia do encapsulamento na programação orientada a objetos é que não seja permitido acessarmos diretamente as propriedades de um objeto.

Encapsulamento

Nesse caso, precisamos operar sempre por meio dos métodos pertencentes a ele. A complexidade de um objeto é escondida, portanto, pela abstração de dados que estão “por trás” de suas operações.

Encapsulamento

Quando aprendemos a criar classes, vimos que na definição de cada método e atributo poderíamos definir um qualificador de acesso, mas foi falado que ainda não era hora de nos preocuparmos com tais detalhes e que poderíamos omitir o qualificador.

Encapsulamento

Pois bem, são exatamente esses qualificadores que nos permitem implementar o encapsulamento.

Encapsulamento

Esses qualificadores nos permitem modificar o nível de acesso aos atributos, aos métodos e até mesmo às classes. São três os possíveis qualificadores de acesso em Java:

Encapsulamento

- *public* (público): indica que o método ou o atributo são acessíveis por qualquer classe, ou seja, que podem ser usados por qualquer classe, independentemente de estarem no mesmo pacote ou estarem na mesma hierarquia;

Encapsulamento

- *private* (privado): indica que o método ou o atributo são acessíveis apenas pela própria classe, ou seja, só podem ser utilizados por métodos da própria classe;

Encapsulamento

- *protected* (protegido): indica que o atributo ou o método são acessíveis pela própria classe, por classes do mesmo pacote ou classes da mesma hierarquia (estudaremos hierarquia de classes quando tratarmos de herança).

Encapsulamento

Quando omitimos o qualificador de acesso, o atributo ou método são considerados protected por padrão.

Encapsulamento

Agora que entendemos o conceito de encapsulamento e conhecemos os três qualificadores de acesso, vem a pergunta: por que restringir o acesso aos atributos de uma classe?

Encapsulamento

Para responder a essa pergunta, utilizaremos como exemplo a classe **Conta** criada em aula anterior.

Encapsulamento

Quando o método *sacar()* daquela classe foi criado, tomamos o cuidado de não permitir saques maiores que o saldo, a fim de que este nunca ficasse negativo.

Encapsulamento

Mas, como não nos preocupamos em restringir o acesso aos atributos daquela classe, um problema que poderia acontecer seria de alguém alterar o saldo, atribuindo um número negativo diretamente ao atributo, como veremos a seguir.

Encapsulamento

```
public static void main(String[] args) {  
    Conta c = new Conta();  
    c.nome_titular = "Ana";  
    c.saldo = -100.0; //Note como é possível atribuir um valor negativo...
```

Encapsulamento

Neste caso, o melhor que teríamos a fazer seria impossibilitar o acesso ao atributo saldo, obrigando que as alterações a ele fossem obrigatoriamente feitas pelos métodos *depositar()* e *sacar()*.

Encapsulamento

De forma geral, a ideia do encapsulamento é a de que cada classe é responsável por controlar seus atributos; portanto, ela deve julgar se aquele novo valor é válido ou não.

Encapsulamento

Essa validação não deve ser controlada por quem está utilizando a classe, e sim pela própria classe, centralizando essa responsabilidade e facilitando futuras mudanças no sistema.

Encapsulamento

Assim, em geral, os atributos de uma classe devem ser privados e deve haver métodos públicos que permitam o acesso a eles.

Exemplo de código a seguir apresenta uma nova versão da classe **Conta** seguindo essa ideia (os construtores não são exibidos na neste exemplo).

Encapsulamento

```
public class Conta {  
    private int numero;  
    private String nome_titular;  
    private double saldo;  
  
    public void depositar(double valor) {  
        this.saldo = this.getSaldo() + valor;  
    }  
  
    public boolean sacar(double valor) {  
        if (this.getSaldo() >= valor) {  
            this.saldo -= valor;  
            return true;  
        }  
        return false;  
    }  
  
    public double getSaldo() {  
        return saldo;  
    }  
  
    public int getNumero() {  
        return numero;  
    }  
  
    public String getNome_titular() {  
        return nome_titular;  
    }  
  
    public void setNome_titular(String nome_titular) {  
        this.nome_titular = nome_titular;  
    }  
}
```


Encapsulamento

Os atributos private de uma classe só podem ser manipulados pelos métodos da classe. Portanto, um cliente de um objeto (isto é, qualquer classe que utilize o objeto) deverá chamar os métodos public da classe para manipular os campos private de um objeto da classe.

Encapsulamento

Por padrão, os atributos “encapsulados” devem ter um método que obtenha o valor atual do atributo (método *get*) e um método que altere o valor do atributo (método *set*).

Encapsulamento

Por exemplo, note que na nova versão da classe **Conta** há um método *getNomeTitular()* que retorna o nome do titular da conta e um método *setNomeTitular(String)* que atribui um novo nome ao titular da conta.

Encapsulamento

Mas, como consideramos que o número da conta é atribuído em sua criação (note que os dois construtores da classe exigem o número) e nunca pode ser alterado, criamos apenas o método *getNumero()*.

Encapsulamento

No caso do saldo, como ele só pode ser alterado por saques e depósitos, não faria sentido criar um método *setSaldo*. Assim, os métodos depositar e sacar servem para alterar o saldo e o *getSaldo()* nos retorna o valor atual do saldo.

Encapsulamento

Vale ressaltar que o qualificador *private* também pode ser usado para modificar o acesso a um método quando este existe apenas para auxiliar a própria classe e não queremos que outras pessoas o utilizem.

Encapsulamento

Entenderemos melhor o uso do qualificador *protected* na próxima aula, quando abordarmos os conceitos de herança e de hierarquias de classes.

Obrigado!